AEGIS LABS
Research Whitepaper

SECURITY RESEARCH

Containing the Simulated Adversary

A Safety Architecture for Autonomous Breach-and-Attack Simulation on Ephemeral Digital Twins

AEGIS Labs Research
Version 1.0 | July 2026

5	default-deny	twin-only
safety invariants enforced in code, not policy	governance gate on every executable action	ephemeral targets, guaranteed teardown, prod refusal

 | github.com/Adversarix/aegis-labs

Table of Contents

1. Executive Summary

Breach-and-attack simulation is moving from human-driven scripts to autonomous agents that reason over MITRE ATT&CK, select techniques, and detonate them through real offensive frameworks. The capability is compelling: an agent can walk the tactics of a campaign adaptively and measure, technique by technique, whether a defender actually detects it. But an autonomous system that executes real adversary behavior is a thing that must itself be contained. The interesting engineering problem is no longer "can the agent run the attack" (existing frameworks already do that) but "can the attack be run repeatedly, unattended, and without ever escaping its box."

This paper describes the safety architecture we developed for exactly that: a small set of invariants, enforced in code rather than asserted in policy documents, that make autonomous emulation safe enough to run continuously. The argument is that safety here is not a wrapper bolted around a dangerous capability. It is the substrate that makes the capability usable at all, because the same properties that keep a simulated adversary contained (a target it cannot leave, an action set it cannot exceed, telemetry that cannot be mistaken for real) are the properties that make its measurements *reproducible*.

> **Key Idea**
>
> Treat the simulator as its own adversary. Every executable action passes through a default-deny governance gate the agent cannot bypass; every target is an ephemeral digital twin the run is guaranteed to destroy; every synthetic telemetry event is cryptographically marked so it cannot be confused with, or forged into, real activity. The core ships in simulation mode: it plans and scores a full campaign without touching anything, and live execution is unlocked only when the invariants are satisfiable.

2. Why Autonomous Emulation Needs a Safety Substrate

A manual purple-team exercise is contained by the human running it. The operator chooses the target, judges each action, and stops when something looks wrong. That containment does not survive automation. An agent that selects and executes techniques on its own removes the human from the inner loop precisely where the judgment used to live, and it does so at machine speed and scale. Three risks follow directly.

- **Blast radius.** A real technique executed against the wrong host is a real intrusion. An agent that mis-resolves a target, or that a misconfiguration points at production, does damage that a report cannot undo.
- **Escalation without a brake.** Adversary emulation is adaptive by design; the point is to chain tactics. An autonomous chainer with no enforced ceiling can walk from benign discovery to destructive action because that is what an adversary would do.
- **Telemetry confusion.** Simulated attacks emit the same signals as real ones. Unmarked, they trigger real incident response, poison detections, and, worse, give a real intruder cover: "it was just the BAS tool."

Manual gating does not scale to this. You cannot put a human in front of every action an agent takes and still call it autonomous. The alternative is to make the unsafe actions *unrepresentable*: to build the system so that the dangerous thing cannot be expressed, not merely discouraged. That is what the following invariants do.

3. A Threat Model for the Simulator Itself

Security tools are usually modeled by what they defend against. An autonomous BAS engine has to be modeled the other way, by what it could do if it went wrong, because it holds live offensive capability. We enumerate the failure modes explicitly and design an invariant against each.

Failure mode	What it looks like	Invariant that prevents it
Wrong target	A real technique runs against a production or third-party host	Target isolation
Out-of-scope action	The agent runs a technique never authorized for this asset class	Default-deny governance
Unreviewed live fire	A destructive step executes with no recorded human decision	Dry-run first
Runaway agent	The loop keeps chaining after something has gone wrong	Kill switch
Telemetry forgery / confusion	Synthetic events are taken for real, or real events hide behind “it’s just a sim”	Signed simulation markers
Orphaned blast radius	A compromised or modified host outlives the run	Ephemeral twin, guaranteed teardown

The rest of the paper takes these one at a time. None is novel in isolation; allowlists, default-deny, and dry-runs are old ideas. The contribution is the claim that *this specific set, enforced in code*, is sufficient to make autonomous emulation safe to run unattended, and the design of the two invariants that are less obvious: signed simulation markers and twin-bounded blast radius.

4. The Five Invariants

The invariants are properties the system holds at runtime, checked in the execution path, not guidelines in a runbook. Each is written so that its violation is an error the code raises, not a judgment call an operator makes.

4.1 Target isolation

Every executable step names a target, and the executor rejects any target host that is not on the run’s twin allowlist. This is the innermost guard: even if every other control were misconfigured, an action cannot reach a host the run did not provision. The check is defense-in-depth by construction: the adapters that wrap real tools re-assert it, so a governance-gate mistake cannot alone put a technique on an unintended host. A technique whose target resolves outside the allowlist does not run; it raises.

4.2 Default-deny governance

Authorization is a positive act. A `(technique, asset-class)` pair that has not been explicitly allowed is denied: silence is refusal, never permission. The governance gate sits in front of execution and the agentic core cannot route around it, because the only path from a plan to a running technique passes through it. This inverts the usual failure mode of automation, where the system does whatever it can unless told to stop; here it does nothing unless told it may.

4.3 Dry-run first

Every technique produces a plan before it produces an effect. In simulation mode the plan is the whole output: a full campaign is reasoned and scored with nothing detonated. Promoting a step to live execution requires a recorded approval tied to that plan, so there is always an auditable answer to “who authorized this action, against

this target, at this time." A live-fire step with no recorded decision behind it is not a policy violation to be caught later; it is a state the runner will not enter.

4.4 Kill switch

A single global control aborts all in-flight steps at the next checkpoint. The property that matters is not that a stop exists (every system can be killed) but that the stop is *clean*: the loop is built to check for it between steps and to hand control to teardown rather than leaving work half-done. An adaptive chainer without a brake is the most dangerous configuration of an autonomous attacker; the kill switch is the brake, and it composes with teardown so that stopping also cleans up.

4.5 Signed simulation markers

This is the least obvious invariant and the one most worth generalizing. Every telemetry event the simulator emits carries a keyed cryptographic tag: an HMAC over the event content, under a per-engagement secret. The tag does two jobs at once.

First, *deconfliction*: the correlation layer and the defender can tell a simulated event from a real one and attribute it to a specific engagement, so a BAS run never launches a real incident response and never silently poisons a detection baseline. Second, *anti-forgery*: because the marker is keyed, merely *observing* a simulated event does not let anything mint new "simulated" events. This closes the ugliest abuse of an unmarked simulator: a real intruder generating events that wear the simulation's clothes so responders wave them through.

Two design notes make the invariant honest. The signing key is per-engagement and must be rotated before any integration with a production SIEM; a marker is only as trustworthy as the secrecy of its key, and a shared or leaked development key is worse than no marker because it invites exactly the forgery it was meant to prevent. And the marker asserts *provenance, not benignity*: it says "this came from the simulator," not "this was harmless." Those are different claims and the system keeps them separate.

5. Ephemeral Digital Twins as Blast-Radius Containment

The invariants above bound *what* runs and *where the signal goes*. Ephemeral digital twins bound *what can be harmed*. Rather than run techniques against the real asset, the harness provisions a disposable stand-in (a twin of the affected asset class), instruments it, runs the technique, captures the result, and destroys it. The target of a real detonation is therefore something whose entire purpose is to be thrown away.

Two properties make the twin a containment boundary rather than merely a test environment.

- **A parity gate.** A twin is only useful if it resembles the asset it stands for; a detection result on a twin that differs materially from the real host is measuring the wrong thing. The fabric refuses to score a run whose twin fails a parity check against the target asset class, so a bad twin produces no verdict rather than a misleading one.
- **Teardown as a guarantee, not a best effort.** The lifecycle is provision, hydrate, instrument, run, capture, *destroy*, and the destroy step is structured as an invariant of the orchestration, executed even when a run fails or is killed. A host that has had a real technique detonated on it is never allowed to outlive the run that created it. This is what lets destructive or self-propagating techniques be exercised at all: their blast radius is the twin, and the twin is guaranteed to cease to exist.

Bounding the blast radius this way is also what makes repeated measurement possible. A test you can only run once, carefully, against production is not a measurement instrument. A test whose target is created and destroyed per run can be executed as often as needed, which is the precondition for treating detection coverage as a number you track rather than an exercise you survive.

6. Independent Observation and Honest Verdicts

Safety and measurement integrity turn out to be the same property viewed twice. A verdict is only honest if the thing observing the attack is independent of the thing executing it; a tool that grades its own homework will report success. The harness therefore records the target's *real* behavior with a sensor separate from the executor, and correlates that observed telemetry against real detection engines rather than trusting the offensive tool's self-report.

The signed-marker invariant is what keeps this correlation clean. Because every synthetic event is attributable, the correlation layer can reason about simulated and real signals without conflating them, and the coverage verdict it produces is a statement about the defender's detections, not about the attacker's claims. Containment and honesty reinforce each other: the same marking that stops a simulated event from triggering a real response also stops it from being silently counted as a real detection.

7. From Safe Execution to Detection Coverage

With execution contained and observation independent, the measurement follows almost as a by-product. Each technique resolves to a verdict under a fixed precedence (*prevented* over *detected* over *logged* over *missed*) and the campaign renders as an ATT&CK coverage map. The precedence matters because it encodes what a defender actually wants to know: that the strongest outcome that occurred is the one reported, so a technique that was merely logged is never credited as detected.

The finding this surfaces, which safer-but-shallower BAS tends to hide, is the *logged-not-detected* gap: the telemetry to catch a technique exists, but no rule fires on it. A tool that reports only "did the test execute" cannot see this gap; a tool that correlates observed telemetry against real detections reports it as the dominant, actionable class of failure. Coverage measurement is treated more fully in a companion note; here the point is narrower: a contained, independently observed simulation is what makes the measurement trustworthy enough to act on.

8. Reproducibility and Responsible Use

Several choices make the architecture a standing instrument rather than a one-off, and keep it on the right side of the line between defensive research and offensive tooling.

- **Simulation by default.** The core plans and scores a full campaign with nothing detonated; live execution is opt-in, gated, and only ever pointed at a twin or a scoped, approved target. The safe path is the default path, and the dangerous path is the one you have to deliberately unlock.
- **Invariants in code, tested.** Target isolation, default-deny, dry-run gating, kill-switch behavior, and marker signing are exercised by tests that run without touching a network, so the safety properties are verified the way the rest of the system is, not left to documentation.
- **Published as method, not as engine.** This paper deliberately describes the safety architecture and the reasoning behind it, not an implementation to run. The techniques used for illustration are benign, already-public atomics; no offensive tradecraft, target data, or engagement results appear here. The contribution we intend to be reusable is the *design discipline*, the specific set of invariants and why each is load-bearing, for anyone building autonomous emulation responsibly.

> **Intended use**
>
> Autonomous breach-and-attack simulation is for authorized security testing only: assessing detections in environments you own or are explicitly permitted to test. The architecture here is defensive in intent and in effect: its entire purpose is to keep a live capability contained. It is described to raise the floor on how such systems are built, not to lower the bar to building them.

9. Conclusion

The hard part of autonomous breach-and-attack simulation is not making an agent attack; it is making an attacking agent safe to run without a human in the loop. We have argued that this reduces to a small set of invariants enforced in code (target isolation, default-deny governance, dry-run-first, a clean kill switch, and signed simulation markers), bounded by ephemeral twins whose destruction is guaranteed. The two invariants worth carrying into other systems are the least obvious: a keyed marker that makes synthetic telemetry attributable and unforgeable, and a disposable target whose teardown is a property of the orchestration rather than a hope. Together they make the same system both contained and honest, and they make its central output, a detection-coverage map you can trust, a thing you can measure repeatedly rather than an exercise you run once and survive.

Citation. Galappatti, K. (2026). *Containing the Simulated Adversary: A Safety Architecture for Autonomous Breach-and-Attack Simulation on Ephemeral Digital Twins.* AEGIS Labs, Adversarix, Inc. github.com/Adversarix/aegis-labs

Scope. This paper describes methodology and safety architecture only; it is not an implementation and contains no offensive tradecraft, target data, or engagement results. Techniques named for illustration are public, benign atomics. MITRE ATT&CK is a trademark of The MITRE Corporation.